Re-implementing LoRA for Parameter-Efficient GPT-2 Adaptation

Edwin Lin, Thomas Peng, Henry Ji CS 4782 Final Project

1. Introduction

Our project studies *LoRA: Low-Rank Adaptation of Large Language Models* by Hu et al. The problem is that full fine-tuning updates every weight in a language model, so each new task needs a large task-specific model copy. LoRA asks whether most of that update can be replaced by a much smaller learned correction.

The method freezes the original model and trains two small matrices, A and B, whose product forms a low-rank update. Low-rank means the update is forced to use a small number of directions, controlled by rank r, rather than changing the whole weight matrix freely. The paper's main contribution is showing that this small update can match full fine-tuning on several NLP tasks while using far fewer trainable parameters.

2. Chosen Result

We reproduced the GPT-2 E2E NLG result from Table 3. This result matters because it tests LoRA on generation, where the model must produce fluent restaurant descriptions from structured slot-value inputs. In the paper, GPT-2 Medium full fine-tuning trains 354.92M parameters and reports BLEU 68.2 and ROUGE-L 71.0. GPT-2 Medium with LoRA trains 0.35M parameters and reports BLEU 70.4 ± 0.1 and ROUGE-L 71.8 ± 0.1.

We reproduced the same comparison at smaller scale because we initially under-estimated the compute needed to exactly replicate the paper: GPT-2 Small full fine-tuning versus LoRA ranks $r \in \{2, 4, 8\}$. We also added a sequential LoRA extension to test whether gradually growing the adapter rank could improve training stability or parameter efficiency, then ran a GPT-2 Medium extension in Colab.

3. Methodology

Model and task. We used GPT-2 Small as an autoregressive language model, meaning it predicts the next token from previous tokens. The task was E2E NLG, a restaurant data-to-text benchmark. Each input is a meaning representation such as restaurant name, food type, and price range. The target is a human-written sentence or short paragraph. We formatted each example as structured input followed by the target text, used the GPT-2 tokenizer, padded with the end-of-sequence token, and capped examples at 256 tokens.

Base LoRA implementation. We implemented LoRA in PyTorch. GPT-2 stores query, key, and value attention projections together in a fused c_attn module, then splits the output into Q, K, and V. Our wrapper keeps the original projection frozen and adds trainable low-rank updates only to the query and value parts. This follows the paper's common setting of adapting Wq and Wv, while keeping the base model fixed. We used $\alpha = r$, so the LoRA scale was 1.

Modifications from the paper. We made four main changes. First, we used GPT-2 Small instead of GPT-2 Medium or Large because we under-estimated the resources needed for exact reproduction. Second, we ran one seed per setting rather than averaging over seeds. Third, we reported BLEU and ROUGE-L only. BLEU measures n-gram overlap with references, while ROUGE-L measures longest shared subsequences. Fourth, we added sequential LoRA. Standard LoRA trains one fixed-rank adapter for the whole run. Sequential LoRA starts with a smaller adapter, then appends new rank-2 adapter stages during training. This lets the model begin with a simple correction and add capacity later. We tested fixed stage boundaries and validation-plateau triggers, and varied whether old stages were frozen, weakly updated, or fully updated.

Training and checks. All runs used 5 epochs, batch size 8, weight decay 0.01, learning rate 2×10^{-4}, and 500 warmup steps in Google Colab. We evaluated generated outputs on the test split. We wrote tests for LoRA injection, freezing behavior, output shape preservation, stage growth, plateau triggering, and merge behavior, with AI tools used to help draft some test cases. These checks mattered because a mistaken freezing rule could silently turn the experiment into full fine-tuning.

4. Results and Analysis

The main result matches the paper's pattern. LoRA reached nearly the same generation quality as full fine-tuning while updating far fewer parameters. LoRA $r = 4$ trained 147,456 parameters, only 0.1184% of the model, and reached BLEU 64.34 and ROUGE-L 67.87. LoRA $r = 8$ trained 294,912 parameters, 0.2364%, and slightly exceeded full fine-tuning on BLEU by 0.45 points. Full fine-tuning remained best on ROUGE-L, with

Experiment	Params	% Total	BLEU	ROUGE-L
Full FT	124.44M	100.0000	64.76	68.24
LoRA $r = 2$	73.7K	0.0592	64.20	67.39
LoRA $r = 4$	147.5K	0.1184	64.34	67.87
LoRA $r = 8$	294.9K	0.2364	**65.21**	67.23

Table 1: GPT-2 Small E2E results. BLEU and ROUGE-L use a 0–100 scale.

68.24 versus 67.23 for LoRA $r = 8$.

We do not treat the LoRA $r = 8$ BLEU score as proof that LoRA is better than full fine-tuning. The gap is small and we only ran one seed. A more careful claim is that LoRA preserved most of the quality while reducing the trainable parameter count by roughly 400x to 1,688x. This is the core tradeoff from the paper.

Extension	Final rank	BLEU	ROUGE-L
Sequential fixed, frozen old	8	64.21	67.65
Sequential fixed, hybrid old	8	64.37	67.02
Sequential fixed, unfrozen old	8	**64.56**	67.37
Sequential plateau, frozen old	8	63.24	67.73
Sequential plateau, hybrid old	8	63.45	**67.77**
Sequential plateau, unfrozen old	8	62.79	67.08

Table 2: Sequential LoRA extension. Each stage added rank 2 until final rank 8.

The sequential extension was useful but did not clearly beat standard LoRA $r = 8$ on GPT-2 Small. Fixed-stage growth performed better than plateau-triggered growth on BLEU. The best sequential BLEU was 64.56, below standard LoRA $r = 8$ at 65.21, while the best sequential ROUGE-L was 67.77, above standard LoRA $r = 8$ at 67.23 but below full fine-tuning at 68.24. This suggests that rank growth is sensitive to scheduling.

The later GPT-2 Medium extension partly changed this picture. LoRA $r = 4$ reached BLEU 65.94 and ROUGE-L 69.25. Plateau sequential LoRA with unfrozen old stages stopped at rank 4 and reached BLEU 65.77 and ROUGE-L 68.12, which supports the idea that adaptive rank growth can rediscover a smaller useful rank. The biggest limitation is still that we used one seed and fewer metrics than the paper.

5. Reflections

The main implementation lesson was that LoRA is simple mathematically but delicate in real model code. GPT-2's c_attn layer required special handling because Q, K, and V are packed together instead of being separate linear layers. Freezing behavior also needed explicit testing. Without it, a run could appear to use LoRA while accidentally training base parameters.

The sequential extension taught us that adding capacity over time is not automatically better. We expected gradual rank growth to work like a curriculum, where the model first learns a simple update and later adds detail. In practice, the fixed schedule was competitive, but the plateau schedule did not improve BLEU. One likely reason is that validation loss plateaus are noisy, so a trigger can add rank at a time that is not actually useful. Another possibility is that old rank directions need to keep adapting after new stages appear.

Given more time, we would run at least three seeds, tune learning rates separately for full fine-tuning, base LoRA, and sequential LoRA, and report confidence intervals. We would also add NIST, METEOR, and CIDEr because the LoRA paper reports them in Table 3 alongside BLEU and ROUGE-L. NIST and METEOR are additional text-overlap metrics, while CIDEr measures agreement with reference descriptions. We would also inspect sample generations for factual slot errors and test sequential LoRA on MLP projections.

6. References

Hu, E. J. et al. (2021). *LoRA: Low-Rank Adaptation of Large Language Models.* arXiv:2106.09685.

Novikova, J., Dusek, O., and Rieser, V. (2017). *The E2E Dataset: New Challenges for End-to-End Generation.* arXiv:1706.09254.

Radford, A. et al. (2019). *Language Models are Unsupervised Multitask Learners.* OpenAI.

Wolf, T. et al. (2020). *Transformers: State-of-the-Art Natural Language Processing.* EMNLP.